

CSCI 240 PA8 Submission

Exercise 1

Source code below:

```
package PA8;

import net.datastructures.TreeMap;
import net.datastructures.Entry;
import net.datastructures.Position;

public class PA8_Ex1 {

    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        TreeMap<Integer, String> bst = new TreeMap<>();

        // 1. Insert 10, 20, 4, 8, 15
        int[] keysToInsert = {10, 20, 4, 8, 15};
        System.out.println("Inserting keys:");
        for (int k : keysToInsert) {
            System.out.println("Inserted: " + k);
            bst.put(k, "Value-" + k);
        }

        // 2. Erase 8, 10
        System.out.println("\nErasing keys:");
        System.out.println("Erasing 8: " + bst.remove(8));
        System.out.println("Erasing 10: " + bst.remove(10));

        // 3. Search for 15, 30, 8
        System.out.println("\nSearching for keys:");
        int[] keysToSearch = {15, 30, 8};
        for (int k : keysToSearch) {
            String val = bst.get(k);
            if (val != null) {
                System.out.println("Found Key " + k + ": " + val);
            } else {
                System.out.println("Key " + k + " NOT Found.");
            }
        }

        // 4. Print the final BST (In-order)
        System.out.println("\nFinal BST Structure:");
        printTreeInOrder(bst, bst.root());
    }

    /**
     * Recursive helper to print the BST in-order.
     */
    private static void printTreeInOrder(TreeMap<Integer, String> map,
        Position<Entry<Integer, String>> p) {
        if (map.isExternal(p)) return;

        printTreeInOrder(map, map.left(p));
```

```
        System.out.println(p.getElement().getKey() + ": " +  
p.getElement().getValue());  
        printTreeInOrder(map, map.right(p));  
    }  
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents  
Modified by: Aiden Wang  
  
--- Inserting Keys ---  
Inserting Key: 10  
Inserting Key: 20  
Inserting Key: 4  
Inserting Key: 8  
Inserting Key: 15  
  
--- Erasing Keys ---  
Erasing 8: Value-8  
Erasing 10: Value-10  
  
--- Searching for Keys ---  
Found Key 15: Value-15  
Key 30 NOT Found.  
Key 8 NOT Found.  
  
--- Final BST Structure (In-order) ---  
Key: 4 Value: Value-4  
Key: 15 Value: Value-15  
Key: 20 Value: Value-20  
  
Process finished with exit code 0
```

Exercise 2

Source code below:

```
package PA8;  
  
import net.datastructures.TreeMap;  
import net.datastructures.Entry;  
import net.datastructures.Position;  
import java.io.File;  
import java.util.Scanner;
```

```
class SimplePopMap {
    private TreeMap<Integer, String> database = new TreeMap<>();

    public SimplePopMap(String filename) {
        try (Scanner sc = new Scanner(new File(filename))) {
            if (!sc.hasNextInt()) return;
            int count = sc.nextInt();
            sc.nextLine(); // consume newline

            for (int i = 0; i < count && sc.hasNextLine(); i++) {
                String line = sc.nextLine();
                int firstComma = line.indexOf(',');
                if (firstComma != -1) {
                    int code = Integer.parseInt(line.substring(0,
firstComma).trim());
                    String data = line.substring(firstComma + 1).trim();
                    database.put(code, data);
                }
            }
            System.out.println("Tree built from " + filename + ". Height:
" + database.height(database.root()));
        } catch (Exception e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }

    public void find(int code) {
        String val = database.get(code);
        if (val != null) {
            System.out.println("Found County " + code + ": " + val);
        } else {
            System.out.println("County " + code + " NOT found.");
        }
    }

    public void insert(int code, String popCounty) {
        if (database.get(code) != null) {
            System.out.println("Replacing County " + code);
        } else {
            System.out.println("Inserting NEW County " + code);
        }
        database.put(code, popCounty);
    }

    public void erase(int code) {
        String removed = database.remove(code);
        if (removed != null) {
            System.out.println("Deleted County " + code);
        } else {
            System.out.println("County " + code + " not found for
deletion.");
        }
    }

    public void print() {
        System.out.println("\nCurrent records (In-order):");
        inOrderPrint(database.root());
    }
}
```

```
        System.out.println("Tree height: " +
database.height(database.root()));
    }

    private void inOrderPrint(Position<Entry<Integer, String>> p) {
        if (database.isExternal(p)) return;
        inOrderPrint(database.left(p));
        System.out.println(p.getElement().getKey() + ": " +
p.getElement().getValue());
        inOrderPrint(database.right(p));
    }
}

public class PA8_Ex2 {
    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        SimplePopMap popMap = new
SimplePopMap("PAs/PA8/Data/popSmall.txt");

        System.out.println("\nRunning tests:");
        System.out.println("1. Initial records:");
        popMap.print();

        System.out.print("\n2. "); popMap.find(6037);
        System.out.print("3. "); popMap.find(6000);

        System.out.print("4. "); popMap.insert(6066, "1, New County, CA");
        System.out.print("5. "); popMap.insert(6065, "2000, Riverside,
CA");

        System.out.print("6. "); popMap.erase(6999);
        System.out.print("7. "); popMap.erase(6075);
        System.out.print("8. "); popMap.erase(6055);

        System.out.println("\n9. Records after modifications:");
        popMap.print();
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java
Modified by: Aiden Wang
```

```
Tree built from PAs/PA8/Data/popSmall.txt. Height: 5
```

```
Running tests:
```

```
1. Initial records:
```

```
Current records (In-order):
```

```
6001: 3648,"Alameda, CA"
6019: 1242,"Fresno, CA"
6037: 22851,"Los Angeles, CA"
6047: 341,"Merced, CA"
6055: 225,"Napa, CA"
6059: 6214,"Orange, CA"
6065: 1784,"Riverside, CA"
6067: 1809,"Sacramento, CA"
6071: 1920,"San Bernardino, CA"
6073: 5351,"San Diego, CA"
6075: 2039,"San Francisco, CA"
6083: 721,"Santa Barbara, CA"
6097: 655,"Sonoma, CA"
```

```
6097: 655,"Sonoma, CA"
6111: 1130,"Ventura, CA"
Tree height: 5
```

```
2. Found County 6037: 22851,"Los Angeles, CA"
3. County 6000 NOT found.
4. Inserting NEW County 6066
5. Replacing County 6065
6. County 6999 not found for deletion.
7. Deleted County 6075
8. Deleted County 6055
```

```
9. Records after modifications:
```

```
Current records (In-order):
```

```
6001: 3648,"Alameda, CA"
6019: 1242,"Fresno, CA"
6037: 22851,"Los Angeles, CA"
6047: 341,"Merced, CA"
6059: 6214,"Orange, CA"
6065: 2000, Riverside, CA
6066: 1, New County, CA
6067: 1809,"Sacramento, CA"
6071: 1920,"San Bernardino, CA"
6073: 5351,"San Diego, CA"
6083: 721,"Santa Barbara, CA"
6097: 655,"Sonoma, CA"
6111: 1130,"Ventura, CA"
Tree height: 5
```

```
Process finished with exit code 0
```

Exercise 3

Source code below:

```
package PA8;

import net.datastructures.AVLTreeMap;
import net.datastructures.Entry;
import net.datastructures.Position;

public class PA8_Ex3 {

    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        AVLTreeMap<Integer, String> avl = new AVLTreeMap<>();

        // 1. Insert 10, 20, 4, 8, 15
        int[] keysToInsert = {10, 20, 4, 8, 15};
        System.out.println("Inserting keys:");
        for (int k : keysToInsert) {
            System.out.println("Inserted: " + k);
            avl.put(k, "Name-" + k);
        }

        // 2. Erase 8, 10
        System.out.println("\nErasing keys:");
        System.out.println("Erasing 8: " + avl.remove(8));
        System.out.println("Erasing 10: " + avl.remove(10));

        // 3. Search for 15, 30, 8
        System.out.println("\nSearching for keys:");
        int[] keysToSearch = {15, 30, 8};
        for (int k : keysToSearch) {
            String val = avl.get(k);
            System.out.println("Key " + k + ": " + (val != null ? "Found"
(" + val + ") " : "NOT Found"));
        }

        // 4. Print the final AVL Tree with Heights
        System.out.println("\nFinal AVL Structure:");
        printAVLInOrder(avl, avl.root());
    }

    private static void printAVLInOrder(AVLTreeMap<Integer, String> tree,
Position<Entry<Integer, String>> p) {
        if (tree.isExternal(p)) return;

        printAVLInOrder(tree, tree.left(p));
        System.out.println("Key: " + p.getElement().getKey() + "\tVal: " +
p.getElement().getValue() + "\tHeight: " + tree.height(p));
        printAVLInOrder(tree, tree.right(p));
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java
Modified by: Aiden Wang

Inserting keys:
Inserted: 10
Inserted: 20
Inserted: 4
Inserted: 8
Inserted: 15

Erasing keys:
Erasing 8: Name-8
Erasing 10: Name-10

Searching for keys:
Key 15: Found (Name-15)
Key 30: NOT Found
Key 8: NOT Found

Final AVL Structure:
Key: 4 Val: Name-4 Height: 1
Key: 15 Val: Name-15 Height: 2
Key: 20 Val: Name-20 Height: 1

Process finished with exit code 0
```

Exercise 4

Source code below:

```
package PA8;

import net.datastructures.AVLTreeMap;
import net.datastructures.Entry;
import net.datastructures.Position;
import java.io.File;
import java.util.Scanner;

/**
 * PA8 - Exercise 4: BetterPopMap (AVL-based Population DB)
 * Modified by: Aiden Wang
 */
class BetterPopMap {
    private AVLTreeMap<Integer, String> database = new AVLTreeMap<>();

    public BetterPopMap(String filename) {
        try (Scanner sc = new Scanner(new File(filename))) {
            if (!sc.hasNextInt()) return;
            int count = sc.nextInt();
            sc.nextLine();
        }
    }
}
```

```
        for (int i = 0; i < count && sc.hasNextLine(); i++) {
            String line = sc.nextLine();
            int firstComma = line.indexOf(',');
            if (firstComma != -1) {
                int code = Integer.parseInt(line.substring(0,
firstComma).trim());
                String data = line.substring(firstComma + 1).trim();
                database.put(code, data);
            }
        }
        System.out.println("AVL Tree built. Height: " +
database.height(database.root()));
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

public void find(int code) {
    String val = database.get(code);
    System.out.println("Search " + code + ": " + (val != null ? val :
"NOT found"));
}

public void insert(int code, String popCounty) {
    if (database.get(code) != null) {
        System.out.println("Replacing County " + code);
    } else {
        System.out.println("Inserting NEW County " + code);
    }
    database.put(code, popCounty);
}

public void erase(int code) {
    String removed = database.remove(code);
    if (removed != null) {
        System.out.println("Deleted County " + code);
    } else {
        System.out.println("County " + code + " not found for
deletion.");
    }
}

public void print() {
    System.out.println("\nPopulation Database (AVL In-order):");
    inOrder(database.root());
    System.out.println("Tree height: " +
database.height(database.root()));
}

private void inOrder(Position<Entry<Integer, String>> p) {
    if (database.isExternal(p)) return;
    inOrder(database.left(p));
    System.out.println(p.getElement().getKey() + ": " +
p.getElement().getValue() + " \t[Height: " + database.height(p) + "]);
    inOrder(database.right(p));
}
}
```

```

public class PA8_Ex4 {
    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        BetterPopMap betterMap = new
BetterPopMap("PAs/PA8/Data/popSmall.txt");

        System.out.println("\nInitial records:");
        betterMap.print();

        System.out.print("\n2. "); betterMap.find(6037);
        System.out.print("3. "); betterMap.find(6000);

        betterMap.insert(6066, "1, New County, CA");
        betterMap.insert(6065, "2000, Riverside, CA");

        betterMap.erase(6999);
        betterMap.erase(6075);
        betterMap.erase(6055);

        System.out.println("\nRecords after modifications:");
        betterMap.print();
    }
}

```

Input/output below:

```

/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -javaagent:/Applicatio
Modified by: Aiden Wang

AVL Tree built. Height: 5

Initial records:

Population Database (AVL In-order):
6001: 3648,"Alameda, CA"    [Height: 1]
6019: 1242,"Fresno, CA"    [Height: 2]
6037: 22851,"Los Angeles, CA" [Height: 1]
6047: 341,"Merced, CA"    [Height: 3]
6055: 225,"Napa, CA"      [Height: 1]
6059: 6214,"Orange, CA"    [Height: 5]
6065: 1784,"Riverside, CA" [Height: 2]
6067: 1809,"Sacramento, CA" [Height: 1]
6071: 1920,"San Bernardino, CA" [Height: 3]
6073: 5351,"San Diego, CA" [Height: 1]
6075: 2039,"San Francisco, CA" [Height: 4]
6083: 721,"Santa Barbara, CA" [Height: 1]
6097: 655,"Sonoma, CA"    [Height: 2]
6111: 1130,"Ventura, CA"   [Height: 1]
Tree height: 5

2. Search 6037: 22851,"Los Angeles, CA"
3. Search 6000: NOT found
Inserting NEW County 6066
Replacing County 6065

```

```

2. Search 6037: 22851,"Los Angeles, CA"
3. Search 6000: NOT found
Inserting NEW County 6066
Replacing County 6065
County 6999 not found for deletion.
Deleted County 6075
Deleted County 6055

Records after modifications:

Population Database (AVL In-order):
6001: 3648,"Alameda, CA"    [Height: 1]
6019: 1242,"Fresno, CA"    [Height: 3]
6037: 22851,"Los Angeles, CA" [Height: 1]
6047: 341,"Merced, CA"    [Height: 2]
6059: 6214,"Orange, CA"    [Height: 5]
6065: 2000, Riverside, CA [Height: 1]
6066: 1, New County, CA    [Height: 3]
6067: 1809,"Sacramento, CA" [Height: 1]
6071: 1920,"San Bernardino, CA" [Height: 2]
6073: 5351,"San Diego, CA" [Height: 4]
6083: 721,"Santa Barbara, CA" [Height: 1]
6097: 655,"Sonoma, CA"    [Height: 2]
6111: 1130,"Ventura, CA"    [Height: 1]
Tree height: 5

Process finished with exit code 0

```

Answer for Question 1

Operations: Insert 10, 20, 4, 8, 15 -> Erase 8 -> Erase 10.

1-5: Insert 10, 20, 4, 8, 15

```

  10
 /  \
4    20
 \   /
  8 15

```

6: Erase 8

```

  10
 /  \
4    20
 /
15

```

Due Date: 4/28/2026

Names: Aiden Wang

7: Erase 10 (Replace with in-order successor, which is 15)

```
  15
 /  \
4    20
```

Final BST:

Tree:

```
  15
 /  \
4    20
```

Answer for Question 2

Operations: Insert 10, 20, 30, 15, 12, 20 (update), Erase 30.

1-3: Insert 10, 20, 30

(Imbalance at 10: RR case -> Single Left Rotation)

```
  20
 /  \
10   30
```

4-5: Insert 15, 12

(Imbalance at 10: RL case -> Double Rotation. Right rotate 15, then Left rotate 10)

```
  20
 /  \
12   30
 /  \
10   15
```

6: Insert 20

(Duplicate key ignored or value replaced; structure remains unchanged)

7: Erase 30

(Imbalance at 20: Left height=1, Right height=-1. BF=2. LL case -> Single Right Rotation)

```
  12
 /  \
10   20
 /
15
```

Final AVL:

Tree:

```
  12
 /  \
10   20
 /
15
```

Extra Credit – Option 1

Source code below:

```
package PA8;

import net.datastructures.TreeMap;
import net.datastructures.AVLTreeMap;
import java.io.File;
import java.util.Scanner;

public class PA8_EC1 {

    public static void main(String[] args) {
        System.out.println("PA8 Extra Credit - Search Tree Performance Analysis\n");

        String[] filenames = {"PAs/PA8/Data/popSmall.txt",
                              "PAs/PA8/Data/popLarge.txt"};

        for (String filename : filenames) {
            System.out.println("--- Analyzing File: " + filename + "
---");

            runBSTAnalysis(filename);
            System.out.println();
            runAVLAnalysis(filename);
            System.out.println();
        }
    }

    private static void runBSTAnalysis(String filename) {
        TreeMap<Integer, String> bst = new TreeMap<>();
        long totalExamined = 0;
        int count = 0;

        try (Scanner sc = new Scanner(new File(filename))) {
            if (sc.hasNextInt()) {
                count = sc.nextInt();
                sc.nextLine();
                for (int i = 0; i < count && sc.hasNextLine(); i++) {
                    String line = sc.nextLine();
                    int comma = line.indexOf(',');
                    if (comma != -1) {
                        int code = Integer.parseInt(line.substring(0,
comma).trim());

                        String data = line.substring(comma + 1).trim();

                        int before = bst.nodesExamined;
                        bst.put(code, data);
                        totalExamined += (bst.nodesExamined - before);
                    }
                }
            }
        }
    }
}
```

```

    }
    }
    double avg = (double) totalExamined / count;
    System.out.println("BST Performance:");
    System.out.println("Nodes inserted: " + count);
    System.out.println("Total node examinations: " +
totalExamined);
    System.out.printf("Average nodes examined per insertion:
%.2f\n", avg);
    } catch (Exception e) {
        System.out.println("BST Error: " + e.getMessage());
    }
}

private static void runAVLAnalysis(String filename) {
    AVLTreeMap<Integer, String> avl = new AVLTreeMap<>();
    long totalExamined = 0;
    int count = 0;

    try (Scanner sc = new Scanner(new File(filename))) {
        if (sc.hasNextInt()) {
            count = sc.nextInt();
            sc.nextLine();
            for (int i = 0; i < count && sc.hasNextLine(); i++) {
                String line = sc.nextLine();
                int comma = line.indexOf(',');
                if (comma != -1) {
                    int code = Integer.parseInt(line.substring(0,
comma).trim());

                    String data = line.substring(comma + 1).trim();

                    int before = avl.nodesExamined;
                    avl.put(code, data);
                    totalExamined += (avl.nodesExamined - before);
                }
            }
        }
        double avg = (double) totalExamined / count;
        System.out.println("AVL Performance:");
        System.out.println("Nodes inserted: " + count);
        System.out.println("Total node examinations: " +
totalExamined);
        System.out.printf("Average nodes examined per insertion:
%.2f\n", avg);
    } catch (Exception e) {
        System.out.println("AVL Error: " + e.getMessage());
    }
}
}

```

Input/output below:

Due Date: 4/28/2026

Names: Aiden Wang

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -javaagent:/Applicati
PA8 Extra Credit - Search Tree Performance Analysis

--- Analyzing File: PAs/PA8/Data/popSmall.txt ---
BST Performance:
Nodes inserted: 14
Total node examinations: 48
Average nodes examined per insertion: 3.43

AVL Performance:
Nodes inserted: 14
Total node examinations: 50
Average nodes examined per insertion: 3.57

--- Analyzing File: PAs/PA8/Data/popLarge.txt ---
BST Performance:
Nodes inserted: 3198
Total node examinations: 5115201
Average nodes examined per insertion: 1599.50

AVL Performance:
Nodes inserted: 3198
Total node examinations: 37479
Average nodes examined per insertion: 11.72

Process finished with exit code 0
```